

Chapter 8. Putting It All Together: Data Processing and Counting Recommender

Now that we have discussed the broad outline of recommender systems, this chapter will put it into a concrete implementation so that we can talk about the choices of technologies and specifics of how the implementation works in real life.

This chapter covers the following topics:

- Data representation with protocol buffers
- Data processing frameworks
- A PySpark sample program
- GloVE embedding model
- Additional foundational techniques in JAX, Flax, and Optax

We will show step-by-step how to go from a downloaded Wikipedia dataset to a recommender system that can recommend words from Wikipedia based on the co-occurrence with words in a Wikipedia article. We use a natural language example because words are easily understood, and their relationships are readily grasped because we can see that related words occur near one another in a sentence. Furthermore, the Wikipedia corpus is easily downloadable and browsable by anyone with an internet connection. This idea of co-occurrence can be generalized to any co-occurring collection of items, such as watching a video in the same session or purchasing cheeses in the same shopping bag.

This chapter will demonstrate concrete implementations of an item-item and a feature-item recommender. Items in this case are the words in an article, and the features are word-count similarity—a MinHash or a kind of locality sensitive hash for words. [Chapter 16](#) covers locality sensitive

hash in more detail, but for now, we'll consider these simple hashing functions to be encoding functions over content, such that content with similar properties maps to similar co-domains. This general idea can be used as a warm-start mechanism on a new corpus in the absence of logging data, and if we have user-item features such as likes, these can be used as features for a feature-item recommender. The principles of co-occurrence are the same, but by using Wikipedia as an example, you can download the data and play with it by using the tools provided.

WARM AND COLD STARTS

A *cold start* occurs when we do not have any information about a corpus or people's preferences and resort to a best-guess approach such as recommending popular items. On the other hand, if items naturally occur in typical groupings, like the selection and arrangement in the cheese aisle of a grocery store, then we call this a warm start: using information like co-occurrence of cheeses with each other or with other items like salami as a means of starting out the recommender engine more intelligently.

In the Wikipedia example, even before we have users click articles, we'll be able to warm-start the word-to-word recommender simply based on how close words are to each other in a sentence. Similarly, if you had a bunch of items that naturally fall into some kind of hierarchical taxonomy, you might be able to warm-start your recommender by having items that are in the same branch of the taxonomy count as co-occurring with one another.

Tech Stack

A set of technologies used together is commonly called a *technology stack*, or *tech stack*. Each component of a tech stack can usually be replaced by other similar technologies. We will list a few alternatives for each component but not go into detail about their pros and cons, as there can be many, and the situation of the deployment will affect the choice of components. For example, your company might already use a particular component, so for familiarity and support, you might wish to use that one.

This chapter covers some of the technology choices for processing the data that goes into building a concrete implementation of a collector.

The sample code is available on [GitHub](#). You might want to clone the code into a local directory.

Data Representation

The first choice of technology we need to make will determine how we represent the data. Some of the choices are as follows:

- [Protocol buffers](#)
- [Apache Thrift](#)
- [JSON](#)
- [XML](#)
- [CSV](#)

In this implementation, we're mostly using protocol buffers because of the ease of specifying a schema and then subsequently serializing and deserializing it.

PROTOCOL BUFFERS

Before protocol buffers were invented, people used to store their binary data in all sorts of custom formats that involved various syntax and specifications (like starting a file with a magic number, followed by rules on how to parse and store various data types like integers, strings, bytes, and floating-point numbers). Protocol buffers unified the storage of custom binary data by allowing users to specify a *schema*, or a named representation of each field and the type of each field (like `first_name` being a string and `age` being an integer). This enables us to easily read and write structured data in a binary format, and the parsing of the data is handled automatically by the protocol buffer library.

For the file format, we're using serialized protocol buffers that are uencoded and written as a single line per record and then bzippped up for

compression. This is just for convenience so that we can parse the files easily without having dependencies on too many libraries. Your company might instead store data in a data warehouse that is accessible by SQL, for example.

Protocol buffers are generally easier to parse and handle than raw data. In our implementation, we will parse the Wikipedia XML into protocol buffers for easier handling using *xml2proto.py*. You can see from the code that XML parsing is a complicated affair, whereas protocol buffer parsing is as simple as calling the `ParseFromString` method, and all the data is then subsequently available as a convenient Python object.

As of June 2022, the Wikipedia dump is about 20 GB in size, and converting to protocol buffer format takes about 10 minutes. Please follow the steps described in the README in the GitHub repo for the most up-to-date steps to run the programs.

In the *proto* directory, take a look at some of the protocol messages defined. This, for example, is how we might store the text from a Wikipedia page:

```
// Generic text document.
message TextDocument {
    // Primary entity, in wikipedia it is the title.
    string primary = 1;
    // Secondary entity, in wikipedia it is other titles.
    repeated string secondary = 2;
    // Raw body tokens.
    repeated string tokens = 3;
    // URL. Only visible documents have urls, some e.g. redirect shouldn't.
    string url = 4;
}
```

The types supported and the schema definitions can be found on the protocol buffer documentation page. This schema is converted into code by using the protocol buffer compiler. This compiler's job is to convert the schema into code that you can call in different languages, which in our case is Python. The installation of the protocol buffer compiler depends

on the platform, and installation instructions can be found in the [protocol buffer documentation](#).

Each time you change the schema, you will have to use the protocol buffer compiler to get a new version of the protocol buffer code. This step can easily be automated by using a build system like Bazel, but this is out of scope for this book. For the purposes of this book, we will simply generate the protocol buffer code once and check it into the repository for simplicity.

Following the directions on the GitHub README, download a copy of the Wikipedia dataset and then run *xml2proto.py* to convert the data to a protocol buffer format. Optionally, use *codex.py* to see what the protocol buffer format looks like. These steps took 10 minutes on a Windows workstation using Windows Subsystem for Linux. The XML parser used doesn't parallelize very well, so this step is fundamentally serial. We'll next discuss how we would distribute the work in parallel either among multiple cores locally or on a cluster.

Big Data Frameworks

The next technology we choose will process data at scale on multiple machines. Some options are listed here:

- [Apache Spark](#)
- [Apache Beam](#)
- [Apache Flink](#)

In this implementation, we're using Apache Spark in Python, or PySpark. The README in the repository shows how to install a copy of PySpark locally using `pip install`.

The first step implemented in PySpark is tokenization and URL normalization. The code is in [tokenize_wiki_pyspark.py](#), but we won't go over it here because a lot of the processing is simply distributed natural language parsing and writing out the data into protocol buffer format. We will instead talk in detail about the second step, which is to make a dictionary of tokens (the *words* in the article) and some statistics about the

word counts. However, we will run the code just to see what the Spark usage experience looks like. Spark programs are run using the program `spark-submit` as follows:

```
bin/spark-submit
--master=local[4]
--conf="spark.files.ignoreCorruptFiles=true"
tokenize_wiki_pyspark.py
--input_file=data/enwiki-latest-parsed --output_file=data/enwiki-latest-tokenize
```

Running the Spark submit script allows you to execute the controller program, in this case, *tokenize_wiki_pyspark.py*, on a local machine as we have in the command line—note that the line `local[4]` means use up to four cores. The same command can be used to submit the job to a YARN cluster for running on hundreds of machines, but for the purposes of trying out PySpark, a decent enough workstation should be able to process all the data in minutes.

This tokenization program converts from a source-specific format (in this case, a Wikipedia protocol buffer) into a more generic text document used for NLP. In general, it's a good idea to use a generic format that all your sources of data can be converted into because that simplifies the data processing downstream. The data conversion can be done from each corpus into a standard format that is handled uniformly by all the later programs in the pipeline.

After submitting the job, you can navigate to the Spark UI (shown in [Figure 8-1](#)) on your local machine at `localhost:4040/stages/`. You should see the job executing in parallel, using up all the cores in your machine. You might want to play with the `local[4]` parameter; using `local[*]` will use up all the free cores on your machine. If you have access to a cluster, you can also point to the appropriate cluster URL.

Stages for All Jobs

Active Stages: 1

▼ Active Stages (1)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Stage Id ▼	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
0	runJob at SparkHadoopWriter.scala:83	+details (kill) 2022/06/13 19:52:54	3.3 min	16/45 (16 running)	517.9 MiB	545.2 MiB		

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Figure 8-1. Spark UI

Cluster Frameworks

The nice thing about writing a Spark program is that it can scale from a single machine with multiple cores to a cluster of many machines with thousands of cores. The full list of cluster types can be found in the [Spark “Submitting Applications” documentation](#).

Spark can run on the following cluster types:

- [Spark Standalone cluster](#)
- [Mesos cluster](#)
- [YARN cluster](#)
- [Kubernetes cluster](#)

Depending on the kind of cluster your company or institution has set up, most of the time submitting the job is just a matter of pointing to the correct URL. Many companies such as Databricks and Google also have fully managed Spark solutions that allow you to set up a Spark cluster with little effort.

PySpark Example

Counting words turns out to be a powerful tool in information retrieval, as we can use handy tricks like term frequency, inverse document frequency (TF-IDF), which is simply the count of words in the documents divided by the number of documents the word has occurred in. This is represented as follows:

$$tfidf_{\text{word}}(i) = \frac{\log_{10} \text{ number of times } word_i \text{ has occurred in corpus}}{\text{number of documents in corpus containing } word_i}$$

For example, because the word *the* appears frequently, we might think it is an important word. But by dividing by the document frequency, *the* becomes less special and drops in importance. This trick is quite handy in simple NLP to get a better-than-random weighting of word importance.

Therefore, our next step is to run [make dictionary.py](#). As the name indicates, this program simply counts the words and documents and makes a dictionary with the number of times a word has occurred.

We have some concepts to cover in order for you to properly grok how Spark helps process data in a distributed manner. The entry point of most Spark programs is `SparkContext`. This Python object is created on the controller. The *controller* is the central program that launches workers that actually process the data. The workers can be run locally on a single machine as a process or on many machines on the cloud as separate workers.

`SparkContext` can be used to create resilient distributed datasets, or RDDs. These are references to data streams that can be manipulated on the controller, and processing on the RDD can be farmed out to all the workers. `SparkContext` allows you to load up data files stored on a distributed filesystem like Hadoop Distributed File System (HDFS) or cloud buckets. By calling the `SparkContext`'s `textFile` method, we are returned a handle to an RDD. A stateless function can then be applied or mapped on the RDD to transform it from one RDD to another by repeatedly applying the function to the contents of the RDD.

For example, this program fragment loads a text file and converts all lines to lowercase by running an anonymous lambda function that converts single lines to lowercase:

```
def lower_rdd(input_file: str,
              output_file: str):
    """Takes a text file and converts it to lowercase."""
    sc = SparkContext()
    input_rdd = sc.textFile(input_file)
    input_rdd.map(lambda line: line.lower()).saveAsTextFile(output_file)
```

In a single-machine implementation, we would simply load up each Wikipedia article, keep a running dictionary in RAM, and count each token and then add 1 to the token count in the dictionary. A *token* is an atomic element of a document that is divided into pieces. In regular English, it would be a word, but Wikipedia documents have other entities such as the document references themselves that need to be kept track of separately, so we call the division into pieces *tokenization* and the atomic elements *tokens*. The single-machine implementation would take a while to go through the thousands of documents on Wikipedia, which is why we

use a distributed processing framework like Spark. In the Spark paradigm, computation is broken into maps, where a function is applied statelessly on each document in parallel. Spark also has a reduce function, where the outputs of separate maps are joined together.

For example, suppose we have a list of word counts and want to sum up the values of words that occur in different documents. The input to the reducer will be something like this:

- (apple, 10)
- (orange, 20)
- (apple, 7)

Then we call the Spark function `reduceByKey(lambda a, b: a+ b)`, which adds all the values with the same key together and returns the following:

- (orange, 20)
- (apple, 17)

If you look at the code in [*make_dictionary.py*](#), the *map phase* is where we take a document as input and then break it into tuples of (token, 1). In the *reduce phase*, the map outputs are joined by the key, which in this case is the token itself, and the reduce function is simply to sum up all the counts of tokens.

Note that the reduce function assumes that the reduction is associative—that is, $(a + b + c) = (a + b) + c = a + (b + c)$. This allows the Spark framework to sum up some parts of the token dictionary in memory on the map phase (in some frameworks, this is called the *combine step*, where you run part of the reduction on the output of the map phase on the mapper machine) and then sum them up over several passes on the reduce phase.

As an optimization, we use the Spark function `mapPartitions`. Map runs the provided function once per line (for which we have encoded an entire Wikipedia document as a protocol buffer, uuencoded as a single text line), whereas `mapPartitions` runs it over an entire partition, which is many documents, usually 64 MB of them. This optimization lets us con-

struct a small Python dictionary over the entire partition so that we have many fewer token-count pairs to reduce. This saves on network bandwidth so the mapper has less data to send to the reducer, and is a good tip in general for these data processing pipelines to reduce network bandwidth (which is generally the most time-consuming part of data processing compared to computation).

Because we are math nerds, and also because group theory shows up a lot in reduction operations, we will briefly introduce an algebraic structure known as *groups* so that you clearly understand all the terms used in the reduction phase.

The concept of sets was mentioned in the introductory chapters; a *set* is a collection of items. The other concept you need to know is an operator. A binary *operator* takes two items and returns another item that is in the set.

Examples of sets that are commonly used are integers, real numbers, and matrices. Examples of binary operators are addition, multiplication, and composition.

An operator and a set denoted by the tuple (binary operator, a set of integers) form a group only if the group axioms are satisfied, namely:

An identity element exists.

For every element x in the group, there exists an element e such that $x + e = e + x = e$. For the addition operation, the identity is 0, and for multiplication, the identity is called 1. This concept is important in the reduction step because in some frameworks the reduction step is initialized with the identity element. For example, sums are usually initialized with 0, and products are usually initialized with 1.

The operator is associative.

For elements x, y, z in the set, $(x + y) + z = x + (y + z)$.

An inverse exists.

For every element x in the group, there exists a y in the group, such that $x + y = e$.

An operator can also be commutative. This isn't a requirement to be a group, but groups that have this property are called *commutative groups*. With commutativity, for elements x, y in the group, $x + y = y + x$. This

property is helpful in the reduction step because it allows the reducer to perform the operations in parallel and then reduce them together without worrying which operations occur in what order.

It is important to note that while addition over real numbers is associative and commutative, addition of floating-point numbers isn't. The reason is that floating-point approximately represents real numbers. So when you add a large number with a small number in floating-point, the small number isn't represented accurately and might simply be discarded. A more accurate and consistent way to add floating-point numbers is to sort the list of numbers to be added first and add all the small numbers up before adding them to the large numbers. Adding two small numbers together first to make a larger number ensures that they do not get lost when being absorbed into the accumulator (the sum). Thus, while addition of numbers is in theory associative and commutative with real numbers, you might get different results in practice with floating-point numbers, depending on the order of operations.

Next we show a complete Spark program that reads in documents in the protocol buffer format of `TextDocument` shown in the preceding code block and then counts how often the words, or tokens, occur in the entire corpus. The file in the GitHub repo is [*make_dictionary.py*](#). The following code is presented slightly differently from the repo file in that it is broken into three chunks for readability and the order of the main and subroutines have been swapped for clarity. Here, we present first the dependencies and flags, then the main body, and then the functions being called by the main body so that the purposes of the functions are clearer.

First, let's look at the dependencies. The main ones are the protocol buffer representing the text document of the Wikipedia article, as discussed earlier. This is the input we are expecting. For the output, we have the `TokenDictionary` protocol buffer, which mainly counts the occurrences of words in the article. We will use the co-occurrences of words to form a similarity graph of articles that we can then use as the basis of a warm-start recommender system. We also have dependencies on PySpark, the data processing framework we are using to process the data, as well as a flag library that handles the options of our program. The `absl` flags library is pretty handy for parsing and explaining the purposes of command-line

flags and also retrieving the set values of flags easily. Here are the dependencies and flags:

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
#
#

"""
    This reads a doc.pb.b64.bz2 file and generates a dictionary.
"""

import base64
import bz2
import nlp_pb2 as nlp_pb
import re
from absl import app
from absl import flags
from pyspark import SparkContext
from token_dictionary import TokenDictionary

FLAGS = flags.FLAGS
flags.DEFINE_string("input_file", None, "Input doc.pb.b64.bz2 file.")
flags.DEFINE_string("title_output", None,
                   "The title dictionary output file.")
flags.DEFINE_string("token_output", None,
                   "The token dictionary output file.")
flags.DEFINE_integer("min_token_frequency", 20,
                    "Minimum token frequency")
flags.DEFINE_integer("max_token_dictionary_size", 500000,
                    "Maximum size of the token dictionary.")
flags.DEFINE_integer("max_title_dictionary_size", 500000,
                    "Maximum size of the title dictionary.")
flags.DEFINE_integer("min_title_frequency", 5,
                    "Titles must occur this often.")

# Required flag.
flags.mark_flag_as_required("input_file")
flags.mark_flag_as_required("token_output")
flags.mark_flag_as_required("title_output")
```

Next, we have the main body of the program, which is where all the sub-routines are called. We first create `SparkContext`, which is the entry

point into the Spark data processing system, and then call its `textFile` method to read in the bziped Wikipedia articles. Please read the README on the repo to understand how it was generated. Next, we parse the text document and send the RDD to two processing pipelines, one to make a dictionary for the body of the article and another to make a dictionary of the titles. We could choose to make a single unified dictionary for both, but having them separate allows us to create a content-based recommender using the token dictionary and an article-to-article recommender using the title dictionary, as titles are identifiers for the Wikipedia article. Here's the main body:

```
def main(argv):
    """Main function."""
    del argv # Unused.
    sc = SparkContext()
    input_rdd = sc.textFile(FLAGS.input_file)
    text_doc = parse_document(input_rdd)
    make_token_dictionary(
        text_doc,
        FLAGS.token_output,
        FLAGS.min_token_frequency,
        FLAGS.max_token_dictionary_size
    )
    make_title_dictionary(
        text_doc,
        FLAGS.title_output,
        FLAGS.min_title_frequency,
        FLAGS.max_title_dictionary_size
    )

if __name__ == "__main__":
    app.run(main)
```

Finally, we have the subroutines called by the main function, all decomposed into smaller subroutines for counting the tokens in the article body and the titles:

```
def update_dict_term(term, dictionary):
    """Updates a dictionary with a term."""
```

```

    if term in dictionary:
        x = dictionary[term]
    else:
        x = nlp_pb.TokenStat()
        x.token = term
        dictionary[term] = x
    x.frequency += 1

def update_dict_doc(term, dictionary):
    """Updates a dictionary with the doc frequency."""
    dictionary[term].doc_frequency += 1

def count_titles(doc, title_dict):
    """Counts the titles."""
    # Handle the titles.
    all_titles = [doc.primary]
    all_titles.extend(doc.secondary)
    for title in all_titles:
        update_dict_term(title, title_dict)
    title_set = set(all_titles)
    for title in title_set:
        update_dict_doc(title, title_dict)

def count_tokens(doc, token_dict):
    """Counts the tokens."""
    # Handle the tokens.
    for term in doc.tokens:
        update_dict_term(term, token_dict)
    term_set = set(doc.tokens)
    for term in term_set:
        update_dict_doc(term, token_dict)

def parse_document(rdd):
    """Parses documents."""
    def parser(x):
        result = nlp_pb.TextDocument()
        try:
            result.ParseFromString(x)
        except google.protobuf.message.DecodeError:
            result = None

```

```

        return result
    output = rdd.map(base64.b64decode)\
        .map(parser)\
        .filter(lambda x: x is not None)
    return output

def process_partition_for_tokens(doc_iterator):
    """Processes a document partition for tokens."""
    token_dict = {}
    for doc in doc_iterator:
        count_tokens(doc, token_dict)
    for token_stat in token_dict.values():
        yield (token_stat.token, token_stat)

def tokenstat_reducer(x, y):
    """Combines two token stats together."""
    x.frequency += y.frequency
    x.doc_frequency += y.doc_frequency
    return x

def make_token_dictionary(
    text_doc,
    token_output,
    min_term_frequency,
    max_token_dictionary_size
):
    """Makes the token dictionary."""
    tokens = text_doc.mapPartitions(process_partition_for_tokens)
        .reduceByKey(tokenstat_reducer).values()
    filtered_tokens = tokens.filter(
        lambda x: x.frequency >= min_term_frequency)
    all_tokens = filtered_tokens.collect()
    sorted_token_dict = sorted(
        all_tokens, key=lambda x: x.frequency, reverse=True)
    count = min(max_token_dictionary_size, len(sorted_token_dict))
    for i in range(count):
        sorted_token_dict[i].index = i
    TokenDictionary.save(sorted_token_dict[:count], token_output)

def process_partition_for_titles(doc_iterator):

```

```

"""Processes a document partition for titles."""
title_dict = {}
for doc in doc_iterator:
    count_titles(doc, title_dict)
for token_stat in title_dict.values():
    yield (token_stat.token, token_stat)

def make_title_dictionary(
    text_doc,
    title_output,
    min_title_frequency,
    max_title_dictionary_size
):
    """Makes the title dictionary."""
    titles = text_doc
        .mapPartitions(process_partition_for_titles)
        .reduceByKey(tokenstat_reducer).values()
    filtered_titles = titles.filter(
        lambda x: x.frequency >= min_title_frequency)
    all_titles = filtered_titles.collect()
    sorted_title_dict = sorted(
        all_titles, key=lambda x: x.frequency, reverse=True)
    count = min(max_title_dictionary_size, len(sorted_title_dict))
    for i in range(count):
        sorted_title_dict[i].index = i
    TokenDictionary.save(sorted_title_dict[:count], title_output)

```

As you can see, Spark makes it easy to scale a program from a single machine to run on a cluster of many machines! Starting from the main function, we create `SparkContext`, read in the input file as a text file, parse it, and then make the token and title dictionaries. The RDD is passed around as arguments of the processing function and can be used multiple times and fed to various map functions (such as the token and title dictionary methods).

The heavy lifting in the make-dictionary methods is done by the process-partitions functions, which are map functions that are applied to entire partitions at once. *Partitions* are large chunks of the input, typically about 64 MB in size and processed as one chunk so that we save on network bandwidth by doing map-side combines. This is a technique to apply the

reducer repeatedly on mapped partitions as well as after joining by the key (which in this case is the token) and summing up the counts. The reason we do this is to save on network bandwidth, which is typically the slowest part of data processing pipelines after disk access.

You can view the output of the `make_dictionary` phase by using the utility `codex.py`, which dumps protocol buffers of different kinds registered in the program. Since all our data is serialized as bziped and uuencoded text files, the only difference is which protocol buffer schema is used to decode the serialized data, so we can use just one program to print out the first few elements of the data for debugging. Although it might be much simpler to store data as JSON, XML, or CSV files, having a schema will save you from future grief because protocol buffers are extensible and support optional fields. They are also typed, which can save you from accidental mistakes in JSON, such as not knowing whether a value is a string or float or int, or having a field as a string in some files and as an int in others. Having an explicit typed schema saves us from a lot of these mistakes.

The next step in the pipeline is `make_cooccurrence.py`. As the name implies, this program simply counts the number of times each token occurs with another token. This is essentially a sparse way of representing a graph. In `nlp.proto`, each row of the sparse co-occurrence matrix is as follows:

```
// Co-occurrence matrix row.
message CooccurrenceRow {
  uint64 index = 1;
  repeated uint64 other_index = 2;
  repeated float count = 3;
}
```

In a *co-occurrence matrix*, each row i has an entry at column j that represents the number of times token j has co-occurred with token i . This is a handy way of associating the similarity between tokens i and j because if they co-occur a lot, they must be more related to each other than tokens that do not co-occur. In the protocol buffer format, these are stored as two parallel arrays of `other_index` and `count`. We use indices because

they are smaller than storing raw words, especially with the varying encoding that protocol buffers use (i.e., the matrix of rows and columns indexed by tokens, and elements that are the co-occurrences of the indices). In this encoding, small integers take fewer bits to represent than large integers; since we reverse-sorted the dictionary by frequency, the most commonly occurring tokens have the smallest indices.

At this stage, if you wanted to make a very simple recommender based on frequent item similarity co-occurrence, you would look up the row for token i and return by count order the tokens j . The simple recommender would make a good variant on the popular item recommender as described in the earlier chapters.

CUSTOMERS ALSO BOUGHT

This concept of co-occurrences will be developed further in [Chapter 9](#), but let's take a moment to reflect on this concept of the MPIR and co-occurrences. When we look at the co-occurrence matrix for items, we can take row sums or column sums to determine the number of times each item has been seen (or purchased). That was how we built the MPIR in [Chapter 2](#). If instead we look at the MPIR for a particular row corresponding to an item the user has seen, that's simply the *conditional MPIR*—i.e., the most popular item, given that the user has seen item i .

However, here we can choose to do an embedding or low-rank representation of the co-occurrence matrix. An embedding representation of a matrix is handy because it allows us to represent each item as a vector. One way to factor the matrix is via singular value decomposition, or SVD (see [“Latent Spaces”](#)), but we won't be doing that here. Instead we will be learning GloVe embeddings, which were developed for NLP.

The objective function of GloVe embedding is to learn two vectors such that their dot product is proportional to the log count of co-occurrence between the two vectors. The reason this loss function works is that the dot product will then be proportional to the log count of co-occurrence; thus, words that frequently occur together will have a larger dot product than words that do not. To compute the embeddings, we need to have the co-occurrence matrix handy, and luckily the previous step in the pipeline has generated such a matrix for us to process.

We introduce feature-item recommenders in this section via the conversion step from words to token IDs. The way we look up the embedding ID for the model is based on the index—either features or items. For the top N popular words, we have a one-to-one mapping from the dictionary index to the embedding ID. However, for long-tailed words, we want them to map to the same value of `embedding_id` if we can help it.

One cheap way of computing a feature from a word is called *min-hashing*: we find 4 consecutive bytes of a word, compute the hash of these bytes, and find the minimum hash of the overlapping 4 bytes. This process makes it more likely to relate *z*e*b*r*a* h*a*s*h*e*s* to zebras. This feature is then used to represent these sets of words as an equivalence class. All words that hash to the same MinHash value are in the same equivalence class. This allows us to handle any new long-tailed word naturally for the time being until a new dictionary is built. It might result in undesirable mistakes in certain applications, but for other applications where it might be safe to do so, a feature-based representation of an item might be mixed into an embedding system as we have done.

Another alternate way to get feature embeddings is to train an autoencoder or some kind of embedding representation that is learned off the features of the item so that the recommender might generalize to new, unseen items. However, for the sake of simplicity and in this word embedding case, we simply use the MinHash for ease of understanding. The MinHash implementation can be seen at [wikipedia/token_dictionary.py](#).

GloVE Model Definition

For this section, please refer to the code at [train_cocurrence.py](#).

Suppose we have tokens i and j from the token dictionary. We know that they have co-occurred with each other N times. We want to somehow generate an embedding space such that the vectors $\mathbf{x}(i) * \mathbf{x}(j)$ are proportional to $\log(N)$. The arguments for log count and the exact equation are

derived in the [“GloVe: Global Vectors for Word Representation”](#) by Jeffrey Pennington et al. We will show just the derived result:

$$y_{\text{predicted}} = x(i) \dot{x}(j) + \text{bias}(i) + \text{bias}(j)$$

Here, x is the embedding lookup. In the code, we use 64-dimensional vectors, which are not too small as to have insufficient capacity to represent the embedding space but are not too large that it would take up too much memory when we have an embedding for the entire dictionary. The bias terms are there to soak up the large counts from very popular items such as *the*, *a*, and *and* that co-occur with many other terms.

The loss we want to minimize is the squared difference between the prediction and the actual value:

$$\begin{aligned} y_{\text{target}} &= 1 + \log_{10}(N) \\ \text{weight} &= \min(1, N/100)^{0.75} \\ \text{loss} &= \text{weight} * (y_{\text{predicted}} - y_{\text{target}})^2 \end{aligned}$$

The weighting term in the loss function is to prevent domination by very popular co-occurrences as well as to downweight rarer co-occurrences.

GloVE Model Specification in JAX and Flax

Let's look at the implementation of the GloVe model based on JAX and Flax. This is in the file *wikipedia/models.py* on the GitHub repository:

```
import flax
from flax import linen as nn
from flax.training import train_state
import jax
import jax.numpy as jnp

class Glove(nn.Module):
    """A simple embedding model based on gloVe.
        https://nlp.stanford.edu/projects/glove/
    """
    num_embeddings: int = 1024
    features: int = 64

    def setup(self):
```

```

self._token_embedding = nn.Embed(self.num_embeddings,
                                  self.features)

self._bias = nn.Embed(
    self.num_embeddings, 1, embedding_init=flax.linen.initializers.zeros)

def __call__(self, inputs):
    """Calculates the approximate log count between tokens 1 and 2.
    Args:
        A batch of (token1, token2) integers representing co-occurrence.
    Returns:
        Approximate log count between x and y.
    """
    token1, token2 = inputs
    embed1 = self._token_embedding(token1)
    bias1 = self._bias(token1)
    embed2 = self._token_embedding(token2)
    bias2 = self._bias(token2)
    dot_vmap = jax.vmap(jnp.dot, in_axes=[0, 0], out_axes=0)
    dot = dot_vmap(embed1, embed2)
    output = dot + bias1 + bias2
    return output

def score_all(self, token):
    """Finds the score of token vs all tokens.
    Args:
        max_count: The maximum count of tokens to return.
        token: Integer index of token to find neighbors of.
    Returns:
        Scores of nearest tokens.
    """
    embed1 = self._token_embedding(token)
    all_tokens = jnp.arange(0, self.num_embeddings, 1, dtype=jnp.int32)
    all_embeds = self._token_embedding(all_tokens)
    dot_vmap = jax.vmap(jnp.dot, in_axes=[None, 0], out_axes=0)
    scores = dot_vmap(embed1, all_embeds)
    return scores

```

Flax is rather simple to use; all networks inherit from Flax's linen neural network library and are modules. Flax modules are also Python data-classes, so any hyper-parameters for the module are defined at the start of the module as variables. We have only two for this simple model: the number of embeddings we want, which corresponds to the number of tokens in the dictionary, and the dimension of the embedding vectors. Next,

in the setup of the module, we actually create the layers we want, which is just the bias term and embedding for each token.

The next part of the definition is the default method that is called when we use this module. In this case, we want to pass in a pair of tokens, i, j ; convert them to embeddings, $\mathbf{x}(i), \mathbf{x}(j)$; and then compute the predicted $\log(\text{count}(y_{\text{predicted}}))$.

In this section of code, we encounter the first difference between JAX and NumPy—namely, a vectorized map, or `vmap`. A `vmap` takes in a function and applies it in the same way across axes of tensors; this makes coding easier because you just have to think about how the original function operates on lower-rank tensors such as vectors. In this example, since we are passing in batches of pairs of tokens and then embedding them, we actually have a batch of vectors, and so we want to run the dot product over the batch dimension. We pass in JAX's dot function, which takes vectors, run it over the batch dimension (which is axis 0), and tell `vmap` to return the outputs as another batch dimension as axis 0. This allows us to efficiently and simply write code for lower-dimensional tensors and obtain a function that can operate on higher-dimensional tensors by `vmap` ping over the extra axes. Conceptually, it would be as if we looped over the first dimension and returned an array of the dot products. However, by converting this process to a function, we allow JAX to push this loop into JITable code that can be compiled to run fast on a GPU.

Finally, we also declare the helper function `score_all`, which takes one token and scores it against all the other tokens. Again, we use `vmap` to take the dot product with the particular token $\mathbf{x}(i)$ but run it against all the other token embeddings. The difference here is that since $\mathbf{x}(i)$ is already a vector, we don't need to `vmap` over it. Therefore, in `in_axes`, we supply `[None, 0]`, which means don't `vmap` over the axes of the first argument but instead `vmap` over axis 0 of the second argument, which is the batch of all the embeddings of all the tokens. Then we return the result, which is an array that is the dot product of $\mathbf{x}(i)$ against all other embeddings but without the bias terms. We don't use the bias term in scoring because it was used in part to soak up the popularity of very common tokens, and our scoring function would be more interesting if we just used the dot product part of it for scoring.

GloVe Model Training with Optax

Next, let's take a look at [wikipedia/train_cooccurrence.py](#). Let's look specifically at the part where the model is called to dig into some JAX specifics:

```
@jax.jit
def apply_model(state, inputs, target):
    """Computes the gradients and loss for a single batch."""

    # Define glove loss.
    def glove_loss(params):
        """The GloVe weighted loss."""
        predicted = state.apply_fn({'params': params}, inputs)
        ones = jnp.ones_like(target)
        weight = jnp.minimum(ones, target / 100.0)
        weight = jnp.power(weight, 0.75)
        log_target = jnp.log10(1.0 + target)
        loss = jnp.mean(jnp.square(log_target - predicted) * weight)
        return loss

    grad_fn = jax.value_and_grad(glove_loss)
    loss, grads = grad_fn(state.params)

    return grads, loss
```

The first point you will notice is the function decorator, `@jax.jit`. This tells JAX that everything in the function is JITable. There are some requirements for a function to be JITable—mostly that it is pure, which is a computer science term indicating that if you call a function with the same arguments, you would expect the same result. That function should not have any side effects and shouldn't rely on a cached state such as a private counter or random-number generator with implicit state. The tensors that are passed in as arguments should probably also have fixed shape, because every new shape would trigger a new JIT compilation. You can give hints to the compiler that certain parameters are constants with `static_argnums`, but these arguments shouldn't change too frequently, or else a lot of time will be spent compiling a program for each of these constants.

One consequence of this pure function philosophy is that the model structure and model parameters are separated. This way, the model functions are pure and the parameters are passed in to the model functions, allowing the model functions to be jitted. This is why we apply the model's `apply_fn` to the parameters rather than simply having the parameters as part of the model.

This `apply_model` function can then be compiled to implement the GloVE loss that we described earlier. The other new functionality that JAX provides above NumPy is automatically computing gradients of functions. The JAX function `value_and_grad` computes the gradient of the loss with respect to the parameters. Since the gradient always points in the direction in which the loss increases, we can use gradient descent to go the other way and minimize the loss. The Optax library has a few optimizers to pick from, including SGD (stochastic gradient descent with momentum) and ADAM.

When you run the training program, it will loop over the co-occurrence matrix and try to generate a succinct form of it by using the GloVE loss function. After about an hour, you should be able to see the highest-scoring term.

The nearest neighbors for “democracy,” for example, are as follows:
democracy:1.064498, liberal:1.024733, reform:1.000746, affairs:0.961664,
socialist:0.952792, organizations:0.935910, political:0.919937,
policy:0.917884, policies:0.907138, and --date:0.889342.

As you can see, the query token itself is usually the highest-scoring neighbor, but this is not necessarily true, as a very popular token might actually be higher scoring to the token than the query token itself.

Summary

After reading this chapter, you should have a good overview of the basic ingredients for assembling a recommender system. You have seen how to set up a basic Python development environment; manage packages; specify inputs and outputs with flags; encode data in various ways, including using protocol buffers; and process the data with a distributed frame-

work with PySpark. You also learned how to compress gigabytes of data into a few megabytes of a model that is able to generalize and quickly score items, given a query item.

Take some time to play with the code and read the documentation of the various packages referenced to get a good sense of the basics. These foundational examples have widespread applications, and having a firm grasp on them will make your production environments more accurate.